

Problemas Racso de Static Trees y Strings

Enunciados interpretados, teoria aplicada, soluciones de examen y pseudocodigo

Como leer este PDF

Este documento toma los problemas de `soluciones_teoria_racso.pdf` que corresponden a la carpeta `statics and strings/`: RMQ, LCA/static trees y strings. No copia literalmente los enunciados de los jueces: cada problema tiene un **enunciado interpretado en espanol**, suficiente para estudiar y responder en papel sin depender de memorizar texto.

El objetivo es que aprendas teoria mediante problemas. En cada seccion veras:

- como interpretar el enunciado;
- que teoria debes activar;
- que estructura usar y por que;
- algoritmo paso a paso;
- pseudocodigo corto estilo C++;
- correctitud;
- complejidad;
- trampas de examen.

Problemas incluidos

Tema	Problemas	Prioridad
RMQ/static array	Static RMQ	Maxima
Strings base	Suffix array y LCP, busqueda, conteo	Maxima
Strings clasicos	distintas, LCS, ordenar substrings, bordes	Maxima/alta
Strings avanzados	Refran, substring periodica, Frequency of String	Alta
Static trees/LCA	Imperial Roads, Fools and Roads	Maxima/alta

Contents

1	Base teorica comun: lo minimo que debes tener en la cabeza	5
1.1	Static significa que puedes preprocesar	5
1.2	RMQ	5
1.3	LCA	5
1.4	Suffix array, rank y LCP	5
1.5	Como clasificar rapido	6

2	Static RMQ	7
2.1	Enunciado interpretado	7
2.2	Como interpretarlo en examen	7
2.3	Teoria necesaria	7
2.4	Algoritmo	7
2.5	Pseudocodigo corto	7
2.6	Correctitud	7
2.7	Complejidad y trampas	8
3	Suffix array y LCP array	9
3.1	Enunciado interpretado	9
3.2	Como interpretarlo en examen	9
3.3	Teoria necesaria	9
3.4	Algoritmo	9
3.5	Pseudocodigo corto	9
3.6	Correctitud	10
3.7	Complejidad y trampas	10
4	Busqueda de subcadenas	11
4.1	Enunciado interpretado	11
4.2	Como interpretarlo en examen	11
4.3	Teoria necesaria	11
4.4	Algoritmo	11
4.5	Pseudocodigo corto	11
4.6	Correctitud	11
4.7	Complejidad y trampas	11
5	Contando subcadenas	12
5.1	Enunciado interpretado	12
5.2	Como interpretarlo en examen	12
5.3	Teoria necesaria	12
5.4	Algoritmo para ocurrencias	12
5.5	Algoritmo para distintas	12
5.6	Pseudocodigo corto	12
5.7	Correctitud	12
5.8	Complejidad y trampas	12
6	Subcadenas diferentes	13
6.1	Enunciado interpretado	13
6.2	Como interpretarlo en examen	13
6.3	Teoria necesaria	13
6.4	Algoritmo	13
6.5	Pseudocodigo corto	13
6.6	Correctitud	13
6.7	Complejidad y trampas	13
7	Subcadena en comun mas larga	14
7.1	Enunciado interpretado	14
7.2	Como interpretarlo en examen	14
7.3	Teoria necesaria	14
7.4	Algoritmo	14
7.5	Pseudocodigo corto	14
7.6	Correctitud	14
7.7	Complejidad y trampas	14

8	Ordenando subcadenas	15
8.1	Enunciado interpretado	15
8.2	Como interpretarlo en examen	15
8.3	Teoria necesaria	15
8.4	Algoritmo	15
8.5	Pseudocodigo corto	15
8.6	Correctitud	15
8.7	Complejidad y trampas	15
9	Bordes	16
9.1	Enunciado interpretado	16
9.2	Como interpretarlo en examen	16
9.3	Teoria necesaria	16
9.4	Algoritmo	16
9.5	Pseudocodigo corto	16
9.6	Correctitud	16
9.7	Complejidad y trampas	16
10	Refran	17
10.1	Enunciado interpretado	17
10.2	Como interpretarlo en examen	17
10.3	Teoria necesaria	17
10.4	Algoritmo	17
10.5	Pseudocodigo corto	17
10.6	Correctitud	17
10.7	Complejidad y trampas	17
11	Substring periodica	18
11.1	Enunciado interpretado	18
11.2	Como interpretarlo en examen	18
11.3	Teoria necesaria	18
11.4	Algoritmo	18
11.5	Pseudocodigo corto	18
11.6	Correctitud	18
11.7	Complejidad y trampas	18
12	Imperial Roads	19
12.1	Enunciado interpretado	19
12.2	Como interpretarlo en examen	19
12.3	Teoria necesaria	19
12.4	Algoritmo	19
12.5	Pseudocodigo corto	19
12.6	Correctitud	19
12.7	Complejidad y trampas	20
13	Fools and Roads	21
13.1	Enunciado interpretado	21
13.2	Como interpretarlo en examen	21
13.3	Teoria necesaria	21
13.4	Algoritmo	21
13.5	Pseudocodigo corto	21
13.6	Correctitud	21
13.7	Complejidad y trampas	22

14 Frequency of String	23
14.1 Enunciado interpretado	23
14.2 Como interpretarlo en examen	23
14.3 Teoria necesaria	23
14.4 Algoritmo con Aho-Corasick	23
14.5 Pseudocodigo corto	23
14.6 Correctitud	23
14.7 Complejidad y trampas	23
15 Ruta de estudio express para estos problemas	24
15.1 Orden recomendado	24
15.2 Plantilla universal de respuesta	24
15.3 Senales para no equivocarte	24

1 Base teorica comun: lo minimo que debes tener en la cabeza

1.1 Static significa que puedes preprocesar

Si el arreglo, arbol o string no cambia, puedes gastar tiempo antes de las consultas. Eso se llama preprocesamiento. En problemas de examen, la frase “muchas consultas” casi siempre significa:

No resuelvas cada consulta desde cero; construye una estructura.

Ejemplos:

- RMQ estatico: sparse table.
- LCA en arbol fijo: binary lifting o Euler tour + RMQ.
- Texto fijo con muchos patrones: suffix array o Aho-Corasick.
- Comparar substrings muchas veces: suffix array + LCP + RMQ.

1.2 RMQ

RMQ significa Range Minimum Query. Dado un arreglo fijo A , una consulta $\text{RMQ}(l, r)$ pide el minimo valor o la posicion del minimo en $A[l..r]$.

La estructura mas directa es sparse table:

$$st[k][i] = \min(A[i], A[i+1], \dots, A[i+2^k-1]).$$

Para consultar $[l, r]$, tomas $k = \lfloor \log_2(r-l+1) \rfloor$ y combinas:

$$\min(st[k][l], st[k][r-2^k+1]).$$

Funciona porque los dos bloques cubren el rango y el minimo es idempotente: $\min(x, x) = x$.

1.3 LCA

El Lowest Common Ancestor de u y v es el ancestro comun mas profundo. Sirve para rutas en arbol:

$$\text{dist}(u, v) = \text{depth}[u] + \text{depth}[v] - 2\text{depth}[\text{lca}(u, v)].$$

Si ademas guardas maximos/minimos en saltos de binary lifting, puedes responder max edge on path o min edge on path.

1.4 Suffix array, rank y LCP

El suffix array de S ordena todos los sufijos:

$$S[\text{SA}[0]..] < S[\text{SA}[1]..] < \dots < S[\text{SA}[n-1]..].$$

El rank inverso dice en que posicion aparece cada sufijo:

$$\text{SA}[\text{rank}[i]] = i.$$

El LCP array guarda:

$$\text{LCP}[i] = \text{lcp}(S[\text{SA}[i-1]..], S[\text{SA}[i]..]).$$

Si dos sufijos estan en posiciones $a < b$ del suffix array:

$$\text{lcp}(i, j) = \min(\text{LCP}[a+1], \dots, \text{LCP}[b]).$$

Por eso se preprocesa RMQ sobre LCP para comparar substrings en $O(1)$.

1.5 Como clasificar rapido

Frase del problema	Estructura que debes pensar
minimo en rango, sin updates	sparse table / RMQ
camino en arbol, distancia, max edge	LCA / binary lifting augmentado
muchos caminos y contar aristas usadas	LCA + diferencia en arbol
buscar patron en texto fijo	suffix array, intervalo de sufijos
subcadenas distintas	total menos suma de LCP
subcadena comun mas larga	SA de $A\#B$ + LCP
comparar/ordenar substrings	rank + LCP arbitrario + RMQ
borde, prefijo=sufijo	KMP prefix function o Z-function
periodicidad	KMP/Z o igualdad por LCP
muchos patrones en un texto	Aho-Corasick o suffix array segun enfoque

2 Static RMQ

Link. <https://codeforces.com/group/Amk1RC8lhZ/contest/694840/problem/A>

2.1 Enunciado interpretado

Se te da un arreglo fijo de n enteros. Luego llegan muchas consultas (l, r) . Para cada una debes responder el minimo del subarreglo desde l hasta r . El arreglo no recibe updates.

2.2 Como interpretarlo en examen

La palabra clave es **static**. Eso descarta segment tree como primera opcion si solo hay minimos y no hay updates. Lo correcto es decir: preproceso bloques de potencias de dos con sparse table y respondo cada rango usando dos bloques solapados.

2.3 Teoria necesaria

Sparse table guarda respuestas para rangos canonicos. Canonico significa: rango que empieza en alguna posicion i y tiene longitud 2^k . La razon de elegir potencias de dos es que cualquier longitud contiene una potencia de dos maxima.

Para $[l, r]$, si $2^k \leq r - l + 1$, entonces:

$$[l, l + 2^k - 1] \quad \text{y} \quad [r - 2^k + 1, r]$$

cubren todo el intervalo. Se pueden solapar. Para minimo eso no afecta, porque repetir elementos no cambia la respuesta.

2.4 Algoritmo

1. Precalcula $lg[len] = \lfloor \log_2 len \rfloor$.

2. Inicializa $st[0][i] = A[i]$.

3. Para cada $k > 0$, calcula:

$$st[k][i] = \min(st[k-1][i], st[k-1][i + 2^{k-1}]).$$

4. Para query (l, r) :

$$k = lg[r - l + 1],$$
$$ans = \min(st[k][l], st[k][r - 2^k + 1]).$$

2.5 Pseudocodigo corto

```
build(A):
    lg[1] = 0
    for i = 2..n: lg[i] = lg[i/2] + 1
    for i = 0..n-1: st[0][i] = A[i]
    for k = 1..lg[n]:
        for i = 0; i + (1<<k) <= n; i++:
            st[k][i] = min(st[k-1][i],
                           st[k-1][i + (1<<(k-1))])

query(l, r):
    k = lg[r-l+1]
    return min(st[k][l], st[k][r-(1<<k)+1])
```

2.6 Correctitud

El preprocesamiento es correcto por induccion en k . Para $k = 0$, cada bloque tiene un solo elemento. Para $k > 0$, un bloque de longitud 2^k es la union de dos bloques de longitud 2^{k-1} , por lo que tomar el minimo de sus respuestas da el minimo del bloque grande.

En la consulta, los dos bloques elegidos cubren $[l, r]$. Todo elemento del rango esta en al menos uno de ellos, asi que el minimo global esta considerado. Si un elemento esta en ambos, no afecta por idempotencia del minimo. Por tanto la respuesta es exactamente el minimo del rango.

2.7 Complejidad y trampas

Preprocesamiento $O(n \log n)$, memoria $O(n \log n)$, query $O(1)$. Trampa: sparse table sirve para min, max y gcd; no sirve así para suma porque dos bloques solapados contarían dos veces.

3 Suffix array y LCP array

Link. <https://codeforces.com/group/AmklRC8lhZ/contest/695758/problem/A>

3.1 Enunciado interpretado

Se te da una cadena S . Debes construir o razonar sobre su suffix array y su LCP array. Este problema es el fundamento de casi todos los demas de strings.

3.2 Como interpretarlo en examen

Si te piden suffix array, no pienses en comparar todos los sufijos con comparacion lenta. Debes explicar una construccion: Manber-Myers si quieres una respuesta simple, o DC3/SA-IS si piden linealidad. Para el examen, Manber-Myers es la ruta mas defendible.

3.3 Teoria necesaria

Un sufijo es $S[i..n-1]$. El suffix array es una permutacion de posiciones. No guarda strings; guarda indices. Si $S = \text{banana}\$,$ el suffix array ordenado es:

$$\text{SA} = [6, 5, 3, 1, 0, 4, 2].$$

El LCP entre sufijos vecinos mide cuanto comparten al inicio. En $\text{banana}\$,$ $\text{ana}\$$ y $\text{anana}\$$ tienen LCP 3.

3.4 Algoritmo

Manber-Myers ordena por prefijos de longitud $1, 2, 4, 8, \dots$

1. Ordena posiciones por caracter y asigna ranks iniciales.
2. En cada ronda de longitud len , representa el sufijo que empieza en i por el par:

$$(\text{rank}[i], \text{rank}[i + len]).$$

3. Ordena esos pares.
4. Reasigna ranks: pares iguales reciben el mismo rank, pares distintos ranks crecientes.
5. Duplica len hasta cubrir toda la cadena.
6. Construye LCP con Kasai usando rank inverso.

3.5 Pseudocodigo corto

```
build_suffix_array(S):
    n = S.size()
    sa = [0,1,...,n-1]
    rank[i] = S[i]
    for len = 1; len < n; len *= 2:
        sort sa by (rank[i], rank[i+len] or -1)
        newRank[sa[0]] = 0
        for t = 1..n-1:
            newRank[sa[t]] = newRank[sa[t-1]]
            if pair(sa[t]) != pair(sa[t-1]):
                newRank[sa[t]]++
        rank = newRank
    return sa

build_lcp(S, sa):
    rank[sa[i]] = i
    h = 0
    for i = 0..n-1:
        if rank[i] == 0: continue
        j = sa[rank[i]-1]
        while S[i+h] == S[j+h]: h++
        lcp[rank[i]] = h
        if h > 0: h--
```

3.6 Correctitud

La induccion es la parte importante. Al inicio, $rank[i]$ ordena prefijos de longitud 1. Si en una ronda $rank$ ordena prefijos de longitud len , entonces el par $(rank[i], rank[i + len])$ ordena prefijos de longitud $2len$: primero mira la mitad izquierda y si empata mira la mitad derecha. Al duplicar hasta $len \geq n$, ordenar prefijos equivale a ordenar sufijos completos.

Kasai es correcto porque al pasar de sufijo i a sufijo $i + 1$, el LCP con su vecino lexicografico disminuye como mucho en uno; por eso reutiliza h y suma tiempo lineal total.

3.7 Complejidad y trampas

Manber-Myers con sort general cuesta $O(n \log^2 n)$; con counting/radix sort por ranks cuesta $O(n \log n)$. Kasai cuesta $O(n)$. Trampa de examen: no confundas SA con $rank$; $SA[pos]$ da indice del sufijo, $rank[i]$ da posicion del sufijo i en el orden.

4 Búsqueda de subcadenas

Link. <https://codeforces.com/group/Amk1RC8lhZ/contest/695758/problem/B>

4.1 Enunciado interpretado

Tienes un texto fijo S y consultas con patrones P . Para cada patron debes decir si aparece, cuantas veces aparece o donde aparece.

4.2 Como interpretarlo en examen

Texto fijo + muchos patrones significa: preprocesa el texto. Con suffix array, un patron aparece exactamente en los sufijos que empiezan con ese patron. Esos sufijos forman un intervalo continuo en el suffix array.

4.3 Teoria necesaria

El orden lexicografico agrupa strings con el mismo prefijo. Si varios sufijos empiezan con P , todos quedan juntos. Por eso el problema se transforma en encontrar el primer y ultimo sufijo cuyo prefijo de longitud $|P|$ coincide con P .

4.4 Algoritmo

1. Construye SA de S .
2. Para cada patron P , haz dos binary searches:
 - lower bound: primer sufijo $\geq P$ lexicograficamente.
 - upper bound: primer sufijo que ya no tiene prefijo P .
3. Si el intervalo $[L, R)$ esta vacio, no aparece.
4. Si no, aparece $R - L$ veces y sus posiciones son $SA[L], \dots, SA[R - 1]$.

4.5 Pseudocodigo corto

```
cmp_suffix_pattern(pos, P):
    // compara S[pos..] con P hasta |P|
    for i = 0..P.size()-1:
        if pos+i == n: return -1
        if S[pos+i] < P[i]: return -1
        if S[pos+i] > P[i]: return 1
    return 0 // P es prefijo del sufijo

find_interval(P):
    L = lower_bound_suffix_geq(P)
    R = first_suffix_greater_than_prefix(P)
    return [L, R)
```

4.6 Correctitud

Si P aparece en posicion i , entonces el sufijo $S[i..]$ empieza con P . Todos los sufijos que empiezan con P tienen el mismo prefijo, asi que en orden lexicografico forman un bloque. Las busquedas binarias encuentran los limites de ese bloque. Por tanto el intervalo contiene exactamente todas las ocurrencias.

4.7 Complejidad y trampas

Construccion $O(n \log n)$. Query simple $O(|P| \log n)$. Trampa: si quieres listar posiciones ordenadas por indice en el texto, el intervalo del suffix array no necesariamente esta ordenado por posicion; debes ordenar esas posiciones o usar otra estructura.

5 Contando subcadenas

Link. <https://codeforces.com/group/Amk1RC81hZ/contest/695758/problem/C>

5.1 Enunciado interpretado

El problema entrena conteos con subcadenas. Segun variante, puede pedir contar ocurrencias de patrones o contar subcadenas que cumplen alguna condicion. La idea base es que toda subcadena es prefijo de algun sufijo.

5.2 Como interpretarlo en examen

Pregunta primero: “me piden contar ocurrencias de un patron” o “me piden contar subcadenas distintas”. Si es patron, usa intervalo en suffix array. Si es distintas, usa suma de LCP.

5.3 Teoria necesaria

Para ocurrencias de P , cuenta cuantos sufijos empiezan con P . Para subcadenas distintas, usa que el sufijo $S[SA[i]..]$ aporta todos sus prefijos, salvo los que comparte con el sufijo anterior.

5.4 Algoritmo para ocurrencias

1. Construye SA.
2. Encuentra intervalo de sufijos que tienen prefijo P .
3. La respuesta es el tamano del intervalo.

5.5 Algoritmo para distintas

$$total = \frac{n(n+1)}{2}, \quad distinct = total - \sum_i LCP[i].$$

5.6 Pseudocodigo corto

```
count_occurrences(P):
    [L, R] = find_interval(P)
    return R - L

count_distinct_substrings():
    ans = n * (n + 1) / 2
    for i = 1..n-1:
        ans -= lcp[i]
    return ans
```

5.7 Correctitud

Para ocurrencias, cada aparicion de P corresponde a un sufijo que empieza con P , y viceversa. Para distintas, cada subcadena se cuenta como prefijo de un sufijo. En orden lexicografico, los prefijos ya vistos por el sufijo actual son exactamente los que comparte con el sufijo anterior, medidos por LCP. Restarlos evita duplicados.

5.8 Complejidad y trampas

Con SA + LCP, el conteo de distintas es $O(n)$. Las consultas de patrones son $O(|P| \log n)$ en version simple. Trampa: $\sum LCP$ no cuenta ocurrencias; cuenta prefijos duplicados entre sufijos vecinos.

6 Subcadenas diferentes

Link. <https://codeforces.com/group/Amk1RC81hZ/contest/695758/problem/D>

6.1 Enunciado interpretado

Dada una cadena S , calcula cuantas subcadenas distintas tiene. Dos subcadenas son iguales si tienen la misma secuencia de caracteres, aunque aparezcan en posiciones distintas.

6.2 Como interpretarlo en examen

Esta es una pregunta casi directa de suffix array + LCP. Si ves “distintas”, piensa en:

$$\frac{n(n+1)}{2} - \sum LCP.$$

6.3 Teoria necesaria

Hay $n(n+1)/2$ subcadenas contando repetidas. Al procesar sufijos en orden de SA, el sufijo actual tiene $n - SA[i]$ prefijos. Sus primeros $LCP[i]$ prefijos ya aparecieron antes, porque los comparte con el sufijo anterior.

6.4 Algoritmo

1. Construye suffix array.
2. Construye LCP array.
3. Inicializa $ans = n(n+1)/2$.
4. Resta cada valor de LCP.

6.5 Pseudocodigo corto

```
distinct_substrings(S):  
    sa = build_sa(S)  
    lcp = build_lcp(S, sa)  
    ans = 1LL * n * (n + 1) / 2  
    for i = 1..n-1:  
        ans -= lcp[i]  
    return ans
```

6.6 Correctitud

Cada subcadena de S es prefijo de algun sufijo. En orden lexicografico, cuando llegas a un sufijo, todos sus prefijos son candidatos a subcadenas nuevas. Pero los prefijos de longitud hasta $LCP[i]$ ya fueron vistos en el sufijo anterior; los prefijos mas largos son nuevos. Por tanto el sufijo aporta $(n - SA[i]) - LCP[i]$ subcadenas nuevas. La suma equivale al total menos $\sum LCP$.

6.7 Complejidad y trampas

Construccion $O(n \log n)$ con Manber-Myers, o $O(n)$ con SA-IS/DC3. Conteo final $O(n)$. Trampa: si la cadena incluye sentinela en la construccion, no debes contar subcadenas que contengan el sentinela.

7 Subcadena en comun mas larga

Link. <https://codeforces.com/group/Amk1RC81hZ/contest/695758/problem/E>

7.1 Enunciado interpretado

Dadas dos cadenas A y B , encuentra la mayor longitud de una subcadena que aparece en ambas, o la subcadena misma.

7.2 Como interpretarlo en examen

Dos strings + subcadena comun = concatena con separadores y usa suffix array. No uses LCS de subsecuencia; aqui es substring, debe ser contigua.

7.3 Teoria necesaria

Si una subcadena X aparece en ambas cadenas, entonces existe un sufijo de A y un sufijo de B que tienen X como prefijo comun. En el suffix array de $A\#B\$,$ los sufijos con prefijo comun largo quedan juntos. Por eso basta mirar LCP entre vecinos de origen distinto.

7.4 Algoritmo

1. Construye $T = A\#B\$,$ con separadores que no aparecen en las cadenas.
2. Construye SA y LCP de T .
3. Define una funcion $origin(pos)$ que diga si el sufijo empieza en A , en B o en separador.
4. Recorre $i = 1..|T| - 1$.
5. Si $SA[i - 1]$ y $SA[i]$ vienen de cadenas distintas, actualiza con $LCP[i]$.

7.5 Pseudocodigo corto

```
longest_common_substring(A, B):
    T = A + "#" + B + "$"
    sa = build_sa(T)
    lcp = build_lcp(T, sa)
    best = 0; bestPos = -1
    for i = 1..T.size()-1:
        if origin(sa[i-1]) != origin(sa[i]):
            if lcp[i] > best:
                best = lcp[i]
                bestPos = sa[i]
    return best
```

7.6 Correctitud

Sea X una subcadena comun optima. Entonces hay al menos un sufijo de A y uno de B que empiezan con X . Todos los sufijos con prefijo X forman un bloque en SA. Dentro de ese bloque debe existir un par adyacente de origen distinto, porque hay sufijos de ambas cadenas. El LCP de ese par es al menos $|X|$. Por tanto el maximo LCP entre vecinos de origen distinto recupera la respuesta optima.

7.7 Complejidad y trampas

$O(n \log n)$ o $O(n)$ segun construccion de SA. Trampa: usa separadores distintos y menores/no presentes; si no, puedes contar substrings que cruzan de A a B artificialmente.

8 Ordenando subcadenas

Link. <https://codeforces.com/group/AmklRC8lhZ/contest/695758/problem/F>

8.1 Enunciado interpretado

Se te da un texto S y una lista de subcadenas representadas por pares (l, r) . Debes compararlas, ordenarlas lexicograficamente o responder cual es menor.

8.2 Como interpretarlo en examen

Comparar substrings directamente cuesta su longitud. Si hay muchas comparaciones, necesitas LCP arbitrario entre sufijos. La estructura es:

$$SA + rank + LCP + RMQ.$$

8.3 Teoria necesaria

Para comparar $S[l_1..r_1]$ y $S[l_2..r_2]$, primero calcula cuanto prefijo comparten:

$$x = lcp(l_1, l_2).$$

Si x alcanza la longitud menor, una substring es prefijo de la otra y gana la mas corta. Si no, el primer caracter distinto esta en $l_1 + x$ y $l_2 + x$.

8.4 Algoritmo

1. Construye SA y $rank$.
2. Construye LCP.
3. Preprocesa sparse table sobre LCP.
4. Para comparar dos substrings, usa RMQ sobre LCP para obtener LCP de los dos sufijos.
5. Decide por longitud o por el primer caracter diferente.

8.5 Pseudocodigo corto

```
lcp_suffix(i, j):
    if i == j: return n - i
    a = rank[i]; b = rank[j]
    if a > b: swap(a, b)
    return rmq_lcp(a+1, b)

less_substring(l1, r1, l2, r2):
    len1 = r1-l1+1
    len2 = r2-l2+1
    x = min(lcp_suffix(l1,l2), min(len1,len2))
    if x == min(len1,len2):
        return len1 < len2
    return S[l1+x] < S[l2+x]
```

8.6 Correctitud

El LCP de los sufijos desde l_1 y l_2 da la longitud del prefijo comun de las dos substrings, salvo que una termine antes. Si el prefijo comun cubre la substring mas corta, entonces la mas corta es menor si las longitudes difieren. Si no cubre, el primer caracter distinto decide exactamente el orden lexicografico.

8.7 Complejidad y trampas

Preprocesamiento $O(n \log n)$ por SA y RMQ. Cada comparacion $O(1)$. Ordenar q substrings cuesta $O(q \log q)$. Trampa: el LCP de sufijos puede ser mas largo que la substring; siempre recortalo con las longitudes.

9 Bordes

Link. <https://codeforces.com/group/Amk1RC81hZ/contest/695758/problem/G>

9.1 Enunciado interpretado

Dada una cadena, encuentra sus bordes o razona con prefijos que tambien son sufijos. Un borde propio no puede ser la cadena completa.

9.2 Como interpretarlo en examen

Prefijo = sufijo es KMP/Z-function. Si el problema pide todos los bordes, piensa en la cadena de prefix function:

$$k = \pi[n-1], \quad k = \pi[k-1], \dots$$

9.3 Teoria necesaria

La prefix function $\pi[i]$ es la longitud del mayor borde de $S[0..i]$. Si $\pi[n-1] = k$, entonces $S[0..k-1]$ es el mayor borde de S . Los bordes mas pequenos son bordes de ese borde.

9.4 Algoritmo

1. Calcula π de KMP.
2. Empieza con $k = \pi[n-1]$.
3. Mientras $k > 0$, registra k y actualiza $k = \pi[k-1]$.
4. Invierte la lista si se pide de menor a mayor.

9.5 Pseudocodigo corto

```
prefix_function(S):
    pi[0] = 0
    for i = 1..n-1:
        j = pi[i-1]
        while j > 0 && S[i] != S[j]:
            j = pi[j-1]
        if S[i] == S[j]: j++
        pi[i] = j

borders(S):
    pi = prefix_function(S)
    k = pi[n-1]
    while k > 0:
        answer.push_back(k)
        k = pi[k-1]
```

9.6 Correctitud

Por definicion, $\pi[n-1]$ es el mayor borde de toda la cadena. Cualquier borde menor de S tambien debe ser borde del mayor borde: si coincide como prefijo y sufijo de S , queda alineado dentro de ese prefijo/sufijo mayor. Por eso seguir $\pi[k-1]$ enumera todos los bordes.

9.7 Complejidad y trampas

$O(n)$. Trampa: revisa si el problema considera la cadena completa como borde. Muchos problemas solo piden bordes propios.

10 Refran

Link. <https://codeforces.com/group/Amk1RC81hZ/contest/695758/problem/H>

10.1 Enunciado interpretado

El problema trata de subcadenas repetidas. Una forma comun es buscar una subcadena que aparece varias veces y maximiza una funcion como $\text{longitud} \times \text{frecuencia}$, o encontrar la repeticion mas significativa.

10.2 Como interpretarlo en examen

Subcadena repetida = varios sufijos con prefijo comun. Varios sufijos con prefijo comun = intervalo en suffix array. Longitud comun de un intervalo = minimo LCP dentro del intervalo.

10.3 Teoria necesaria

Si tienes t sufijos consecutivos en SA, la longitud maxima que comparten todos es:

$$\min(\text{LCP}[L+1], \dots, \text{LCP}[R]).$$

Si esa longitud es h , entonces existe una subcadena de longitud h que aparece al menos $t = R - L + 1$ veces.

10.4 Algoritmo

Interpreta el arreglo LCP como histograma:

- una barra de altura h representa que dos sufijos vecinos comparten h ;
- un intervalo donde todas las barras son al menos h representa muchos sufijos compartiendo un prefijo de longitud h ;
- si el intervalo tiene w barras, hay $w + 1$ sufijos.

Con una pila monotona encuentras para cada LCP el intervalo maximal donde es el minimo. Evalua la funcion del problema.

10.5 Pseudocodigo corto

```
best_repeated():
    sa = build_sa(S)
    lcp = build_lcp(S, sa)
    stack = empty
    best = 0
    for i = 1..n: // usa lcp[n]=0 como centinela
        start = i
        while stack not empty && stack.top.height > lcp[i]:
            (h, pos) = stack.pop()
            width = i - pos
            freq = width + 1
            best = max(best, h * freq)
            start = pos
        push (lcp[i], start)
```

10.6 Correctitud

Cada subcadena repetida corresponde a un conjunto de sufijos consecutivos que la tienen como prefijo. La frecuencia es el numero de sufijos del bloque y la longitud maxima comun es el minimo LCP del bloque. La pila monotona enumera para cada posicion de LCP el mayor intervalo donde esa altura es el minimo. Por tanto considera todos los bloques candidatos relevantes.

10.7 Complejidad y trampas

SA + LCP domina; la pila es $O(n)$. Trampa: si el problema pide devolver la subcadena, no basta el valor; debes guardar una posicion de sufijo dentro del intervalo ganador.

11 Substring periodica

Link. <https://codeforces.com/group/Amk1RC81hZ/contest/695758/problem/I>

11.1 Enunciado interpretado

Se trabaja con substrings y periodos. Debes decidir si una substring puede describirse como repeticiones de un bloque mas corto, o contar/optimizar substrings periodicas.

11.2 Como interpretarlo en examen

Periodicidad significa igualdad entre una substring y ella misma desplazada. Para muchas queries de igualdad, usa LCP arbitrario:

$$lcp(l, l + p) \geq len - p.$$

11.3 Teoria necesaria

Una cadena X de longitud m tiene periodo p si:

$$X[i] = X[i + p] \quad \text{para } 0 \leq i < m - p.$$

Eso equivale a decir que el prefijo de longitud $m - p$ coincide con el bloque desplazado p posiciones.

11.4 Algoritmo

Para una query substring $S[l..r]$ de longitud m y candidato p :

1. Calcula $x = lcp(l, l + p)$ con SA + LCP + RMQ.
2. Si $x \geq m - p$, entonces p es periodo.
3. Si ademas el problema pide repeticion exacta de bloques, revisa $m \bmod p = 0$.

Para buscar periodos, se prueban divisores o candidatos derivados de KMP/Z, segun la forma exacta del problema.

11.5 Pseudocodigo corto

```
is_period(l, r, p):
    m = r - l + 1
    if p <= 0 || p >= m: return false
    return lcp_suffix(l, l+p) >= m-p

is_exact_repetition(l, r, p):
    m = r - l + 1
    return m % p == 0 && is_period(l, r, p)
```

11.6 Correctitud

La definicion de periodo exige que cada caracter coincida con el que esta p posiciones despues mientras ambos existan. Eso es exactamente comparar $S[l..r - p]$ con $S[l + p..r]$. Si el LCP entre los sufijos que empiezan en l y $l + p$ alcanza $m - p$, esas dos substrings son iguales, por lo tanto p es periodo. Si no alcanza, hay una posicion que viola la definicion.

11.7 Complejidad y trampas

Preprocesamiento $O(n \log n)$. Cada prueba de periodo $O(1)$. Trampa: “tener periodo p ” no siempre significa que la longitud sea multiplo de p ; para repeticion exacta si debes pedir divisibilidad.

12 Imperial Roads

Link. <https://codeforces.com/gym/101889/problem/I>

12.1 Enunciado interpretado

Se tiene un grafo ponderado y se consideran consultas sobre carreteras. Para una carretera dada, se quiere saber el costo minimo de un arbol generador que la incluya obligatoriamente.

12.2 Como interpretarlo en examen

Este es un problema de MST + LCA/static tree. Primero construyes un MST. Luego cada consulta pregunta: si fuerzo una arista (u, v, w) , que arista saco del ciclo que se forma? La respuesta es sacar la arista mas pesada del camino entre u y v en el MST.

12.3 Teoria necesaria

Propiedad del ciclo del MST: si agregas una arista a un arbol, aparece un unico ciclo. Para volver a tener un arbol, debes eliminar una arista de ese ciclo. Si la arista nueva es obligatoria, la mejor opcion es eliminar la mas pesada del camino original.

Para responder maximo en camino rapido, usa binary lifting augmentado:

$$up[v][j] = \text{ancestro } 2^j,$$

$$mx[v][j] = \text{maximo peso en ese salto.}$$

12.4 Algoritmo

1. Construye MST con Kruskal y guarda su peso total W .
2. Enraiza el MST y preprocesa up y mx .
3. Para consulta (u, v, w) , calcula $M = \text{maxEdge}(u, v)$.
4. Responde:

$$W + w - M.$$

12.5 Pseudocodigo corto

```
maxEdge(u, v):
    ans = 0
    if depth[u] < depth[v]: swap(u, v)
    lift u to depth[v], updating ans with mx[u][j]
    if u == v: return ans
    for j = LOG-1 downto 0:
        if up[u][j] != up[v][j]:
            ans = max(ans, mx[u][j], mx[v][j])
            u = up[u][j]; v = up[v][j]
    ans = max(ans, mx[u][0], mx[v][0])
    return ans

answer(edge u,v,w):
    return mstWeight + w - maxEdge(u,v)
```

12.6 Correctitud

Agregar la arista consultada al MST crea un ciclo formado por esa arista y el camino $u - v$ dentro del MST. Todo arbol generador que incluya la arista consultada debe eliminar alguna arista de ese ciclo. Para minimizar el peso total, se elimina la arista mas pesada del camino original. La tabla mx calcula exactamente esa arista maxima al descomponer el camino en saltos de potencias de dos.

12.7 Complejidad y trampas

Kruskal $O(m \log m)$. LCA $O(n \log n)$ memoria y preproceso. Cada query $O(\log n)$. Trampa: si la arista ya pertenece al MST, la formula sigue dando W porque el maximo del camino puede ser la misma arista con peso w .

13 Fools and Roads

Link. <https://codeforces.com/problemset/problem/191/C>

13.1 Enunciado interpretado

Se da un arbol con n nodos y luego muchas rutas entre pares de nodos. Para cada arista del arbol, debes contar cuantas de esas rutas pasan por ella.

13.2 Como interpretarlo en examen

Muchas rutas sobre arbol + contar aristas = diferencia en arbol. No actualices cada camino arista por arista. Marca extremos y LCA, luego acumula con DFS.

13.3 Teoria necesaria

En un arreglo, para sumar 1 en un intervalo $[l, r]$, haces:

$$diff[l] ++, \quad diff[r + 1] --.$$

En arbol, una ruta $u - v$ se parte en $u \rightarrow a$ y $v \rightarrow a$, donde $a = lca(u, v)$. La version arborea es:

$$cnt[u] ++, \quad cnt[v] ++, \quad cnt[a] -- = 2.$$

Luego cada subarbol acumula hacia arriba.

13.4 Algoritmo

1. Preprocesa LCA.

2. Para cada ruta (u, v) :

$$cnt[u] ++, \quad cnt[v] ++, \quad cnt[lca(u, v)] -- = 2.$$

3. Haz DFS postorden desde la raiz.

4. Cuando vuelves de un hijo x , su acumulado final es la cantidad de rutas que pasan por la arista $(parent[x], x)$.

13.5 Pseudocodigo corto

```
for each path (u, v):
    a = lca(u, v)
    cnt[u]++
    cnt[v]++
    cnt[a] -= 2

dfs(u, p):
    sum = cnt[u]
    for v in adj[u]:
        if v == p: continue
        child = dfs(v, u)
        answer_edge(u,v) = child
        sum += child
    return sum
```

13.6 Correctitud

Para una ruta $u - v$, los $+1$ en u y v suben por el arbol durante la acumulacion. Al llegar al LCA, restar 2 cancela esas dos contribuciones antes de que pasen por encima del LCA. Por tanto solo quedan marcadas las aristas de los caminos $u \rightarrow a$ y $v \rightarrow a$, que juntos forman exactamente la ruta $u - v$. Como las contribuciones se suman linealmente, el metodo funciona para todas las rutas a la vez.

13.7 Complejidad y trampas

LCA $O(n \log n)$. Cada ruta $O(\log n)$. DFS $O(n)$. Trampa: el valor de una arista se guarda normalmente en el hijo, no en el padre.

14 Frequency of String

Link. <https://codeforces.com/problemset/problem/963/D>

14.1 Enunciado interpretado

Hay un texto fijo y varias consultas. Cada consulta da un patron P y un numero k . Se pide la menor longitud de una substring del texto que contenga al menos k ocurrencias de P . Si no hay suficientes ocurrencias, la respuesta es imposible o -1 segun formato.

14.2 Como interpretarlo en examen

Primero reduce el problema: si conoces las posiciones donde aparece P , la respuesta es una ventana sobre esas posiciones. La parte dificil es obtener ocurrencias de muchos patrones. Para muchos patrones, Aho-Corasick es natural; con suffix array tambien puedes localizar ocurrencias como intervalo.

14.3 Teoria necesaria

Si P aparece en posiciones:

$$p_1 < p_2 < \dots < p_t,$$

entonces una substring que contiene las ocurrencias $p_i, \dots, p_{i+k-1}$ debe empezar no despues de p_i y terminar al menos en $p_{i+k-1} + |P| - 1$. La menor ventana para ese grupo tiene longitud:

$$p_{i+k-1} - p_i + |P|.$$

Por tanto basta revisar ocurrencias consecutivas.

14.4 Algoritmo con Aho-Corasick

1. Inserta todos los patrones en el trie.
2. Construye failure links.
3. Recorre el texto una vez.
4. Cada vez que un patron termina en la posicion actual, guarda su posicion de inicio.
5. Para cada consulta (k, P) , toma la lista de posiciones de P y calcula el minimo de $pos[i+k-1] - pos[i] + |P|$.

14.5 Pseudocodigo corto

```
solve_query(pattern_id, k):
    v = occurrences[pattern_id]
    if v.size() < k: return -1
    ans = INF
    len = pattern_length[pattern_id]
    for i = 0; i+k-1 < v.size(); i++:
        ans = min(ans, v[i+k-1] - v[i] + len)
    return ans
```

14.6 Correctitud

Toda substring que contiene k ocurrencias tiene una primera ocurrencia y una k -esima ocurrencia dentro de ella. Si esas ocurrencias son p_i y p_{i+k-1} , la substring debe cubrir desde p_i hasta el final de la ocurrencia en p_{i+k-1} . La ventana minima para ese conjunto tiene longitud $p_{i+k-1} - p_i + |P|$. Revisar todos los grupos consecutivos de k ocurrencias cubre todas las respuestas posibles.

14.7 Complejidad y trampas

Construir Aho-Corasick cuesta $O(L|\Sigma|)$ o $O(L)$ segun implementacion, donde L es la suma de longitudes de patrones. Recorrer texto $O(n + occ)$. Cada query cuesta proporcional al numero de ocurrencias del patron si se hace directo. Trampa: ocurrencias pueden solaparse; no debes descartarlas.

15 Ruta de estudio express para estos problemas

15.1 Orden recomendado

1. Static RMQ: aprende sparse table hasta poder probarla sin mirar.
2. Suffix array + LCP: entiende suffix, rank, LCP y Manber-Myers.
3. Búsqueda de subcadenas: patron como intervalo en SA.
4. Subcadenas distintas y LCS: formulas classicas con LCP.
5. Ordenando subcadenas: LCP arbitrario con RMQ.
6. Bordes: prefix function de KMP.
7. Refran/periodicidad/Frequency: problemas avanzados que combinan ideas.
8. Imperial/Fools: LCA aplicado a caminos y diferencias en arbol.

15.2 Plantilla universal de respuesta

Cuando te den un problema de este bloque, responde asi:

1. Identifico el objeto fijo: arreglo, arbol o string.
2. Como hay muchas consultas, preproceso.
3. Defino la estructura y que guarda.
4. Transformo la query a una operacion de esa estructura.
5. Pruebo correctitud con el invariante clave.
6. Cierro complejidad de preproceso, query y memoria.

15.3 Senales para no equivocarte

- “No updates” + rango = sparse table.
- “Camino en arbol” = LCA, a veces con max/min por salto.
- “Muchas rutas y contar aristas” = diferencia en arbol.
- “Patron en texto fijo” = intervalo en suffix array.
- “Distinct substrings” = $n(n+1)/2 - \sum LCP$.
- “Comparar substrings” = LCP arbitrario + RMQ.
- “Prefijo y sufijo” = KMP/Z.
- “Muchas ocurrencias de muchos patrones” = Aho-Corasick o SA.